

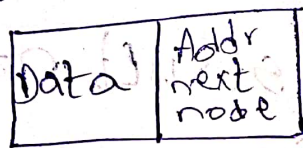
Single Linked List

08/04/25

* Linked List is a linear data structure

* In Linked List each element is called as node. The nodes are not stored in continuous memory and each node is connected with reference point called address.

* The node contains data & address of next node.



* In linked list, the first node is called as head node. The last node's next part is NULL.

Ex:- structure

struct node

{
int data;

struct node *next; self referential structure
}*head = NULL;

* The structure variable is a pointer and it is a member of the structure is called as self referential structure.

* (.) , (→) are called as member access operators.

Insert Last

```
#include <stdio.h>
struct node {
    int data;
    struct node *next;
} *head = NULL;

void insertlast (int val) {
    struct node *newnode;
    newnode = (struct node *) malloc
        (sizeof (struct node));
    newnode->data = val;
    newnode->next = NULL;
    if (head == NULL) {
        head = newnode;
    }
    else {
        struct node *t = head;
        while (t->next != NULL) {
            t = t->next;
        }
        t->next = newnode;
    }
}
```

```

void display() {
    if (head == NULL)
        printf("SLL is empty\n");
    else
    {
        struct node *t = head;
        while (t != NULL)
        {
            printf("%d -> ", t->data);
            t = t->next;
        }
    }
}

```

```

- }
printf("NULL\n");

```

```

}

```

21/01/25

Insert First

void insertfirst(int val)

```

{
    struct node *newnode;
    newnode = (struct node*) malloc
    (sizeof(struct node));

```

```

    newnode->data = val;

```

```

    newnode->next = NULL;

```

```

    if (head == NULL)
    {

```

```

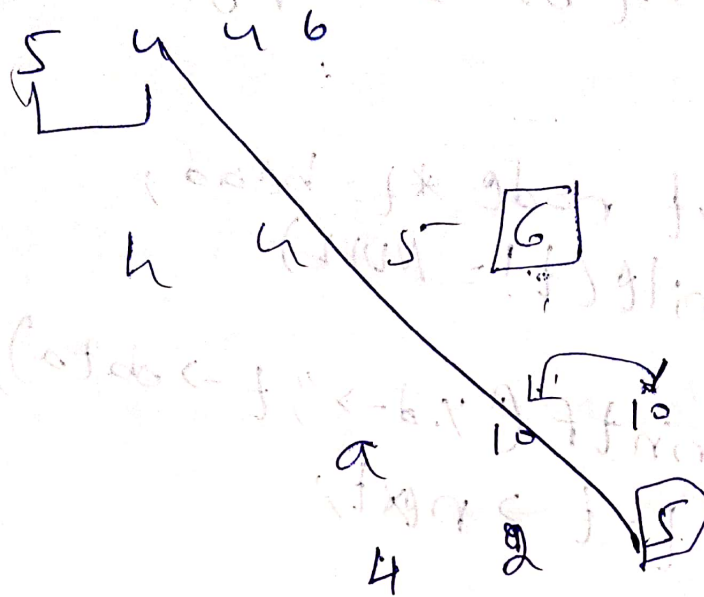
        head = newnode;
    }

```

```

    else
    {

```



```

newnode → next = head;
head = newnode;

```

```

}

```

```

}

```

Insert At Middle

```

void insertMiddle (int val, int pos)

```

```

{

```

```

    int i;

```

```

    struct node * newnode;

```

```

    newnode = (struct node *) malloc (sizeof (struct node));

```

```

    newnode → data = val;

```

```

    newnode → next = NULL;

```

```

    struct node * p = head;

```

```

    for (i = 1; i < pos; i++) {

```

```

        p = p → next;

```

```

    }

```

```

}

```

$q \rightarrow next = newnode;$
 $newnode \rightarrow next = P;$

}

Delete first

void deleteFirst()

{ if (head == NULL);
printf("SLL is empty");

else

{ struct node *t = head;
head = head->next;
free(t);

}

}

Delete Last

void deleteLast()

{ if (head == NULL)
printf("SLL is empty");
else if (head->next == NULL)

{ struct node *t = head;
head = head->next;
free(t);

}

else

{

```

struct node *p = head; *a;
while (p->next != NULL) {
    a = p;
    p = p->next;
}
a->next = NULL;
free(p);
}
}

```

Delete middle

22/02/25

```

void deleteMiddle(int pos)
{
    struct node *p = head, *a;
    for (int i = 1; i < pos; i++)
    {
        a = p;
        p = p->next;
    }
    a->next = p->next;
    free(p);
}

```

searching a Node
void search (int key)

{

struct node * t;

for (t = head; t != NULL; t = t → next)

{

if (t → data == key)

{
printf ("key found");

break;

}

}

if (t == NULL)

printf ("key not found");

}

Counting Nodes

void count ()

{ int count = 0;
if (head == NULL)

{ printf ("SLL is empty");

}

else

{

struct node * t = head;

while (t != NULL)

{

count ++;

t = t → next;

}

```
printf("%.1d", (int));
```

```
}
```

```
}
```

```
void findMax ( )
```

```
{
```

```
struct node* t;
```

```
max = head int max = head->data;
```

```
for (t = head; t != NULL; t = t->next)
```

```
{
```

```
if (maxtdata > t->next > data)
```

```
{
```

```
printf("%.1d", t);
```

```
max = t > t->data;
```

```
}
```

```
}
```

```
printf("%.1d", max);
```

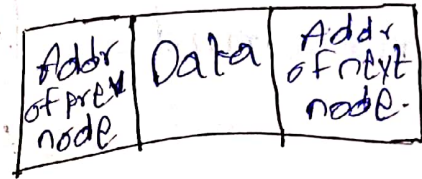
```
}
```

Double Linked List (DLL)

[24/12/21]

In DLL every node is maintaining address of next node and previous node.

DLL node structure



struct node {

int data;

struct node *prev;

struct node *next;

*head = NULL;

Insert First

void insertFirst (int val) {

struct node *newnode;

newnode = (struct node *) malloc
(sizeof (struct node));

newnode → data = val;

newnode → prev = NULL;

newnode → next = NULL;

if (head == NULL)

head = newnode;

else

{

newnode → next = head;

head → prev = newnode;

head = newnode;

}

}

Insert Last

```
void insertLast(int val) {  
    struct node * newnode;  
    newnode = (struct node *) malloc(  
        (sizeof (struct node)));  
    newnode -> data = val;  
    newnode -> prev = NULL;  
    newnode -> next = NULL;  
    if (head == NULL)  
        head = newnode;  
    else  
    {  
        struct node * t = head;  
        while (t -> next != NULL) {  
            t = t -> next;  
            t -> next = newnode;  
            newnode -> prev = t;  
        }  
    }
```

Insert Middle

```
void insertMiddle(int val, pos) {  
    struct node * newnode;  
    newnode = (struct node *) malloc (sizeof  
        (struct node));
```

```
newnode → data = val;  
newnode → prev = NULL;  
newnode → next = NULL;  
if (head == NULL)
```

```
    head = newnode;
```

```
else
```

```
    struct node *p = head, a;
```

```
    for (int i = 1; i < pos; i++) {
```

```
        a = p;
```

```
        p = p → next;
```

```
    }  
    a → next = newnode;
```

```
    p → prev = newnode;
```

```
    newnode → prev = a;
```

```
    newnode → next = p;
```

```
}
```

Delete Last

```
void deleteLast () {
```

```
    if (head == NULL)
```

```
        printf("DLL is empty\n");
```

```
    else if (head → next == NULL)
```

```
        struct node *t = head;
```

```
        head = NULL;
```

```
        free(t);
```

```
    else {
```

```
        struct node *t = head;
```

```
while (t → next != NULL)
```

```
t = t → next;
```

```
t → prev → next = NULL;
```

```
free(t)
```

```
}  
}
```

Delete First

```
void deleteFirst() {
```

```
if (head == NULL)
```

```
printf("DLL is empty \n");
```

```
else if (head → next == NULL) {
```

```
struct node *t = head;
```

```
head = NULL;
```

```
free(t);
```

```
}
```

```
else
```

```
{
```

```
struct node *t = head;
```

```
head = head → next;
```

```
head → prev = NULL;
```

```
free(t);
```

```
}
```

```
}
```

Delete Middle

```
void deleteMiddle (int pos){
```

```
    struct node * t = head;
```

```
    for (int i = 1; i < pos; i++)
```

```
        t = t->next;
```

```
    t->prev->next = t->next;
```

```
    t->next->prev = t->prev;
```

Display

```
void display () {
```

```
    if (head == NULL)
```

```
        printf("DLL is empty\n");
```

```
    else
```

```
    {
```

```
        struct node * t = head;
```

```
        while (t != NULL)
```

```
        {
```

```
            printf("%d -> NULL\n", t->data);
```

```
            t = t->next;
```

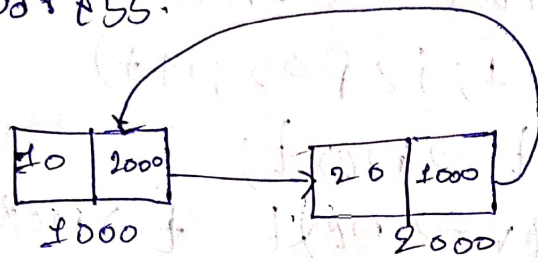
```
        }
```

```
    }
```

```
}
```

Circular Linked List

In C.L.L the next part ~~is~~ refers to head address.



Node structure:-

```
struct node
{
    int data;
    struct node *next;
} *head = NULL;
```

Insert First

```
void insertFirst(int val)
{
    struct node *newnode;
    newnode = (struct node*) malloc (sizeof(
        struct node));
    newnode->data = val;
    newnode->next = NULL;
    if (head == NULL)
    {
        head = newnode;
        head->next = head;
    }
    else
    {
        struct node *t = head;
```

```

while (t->next != head) {
    t = t->next;
}
newnode->next = head;
head = newnode;
t->next = head;
}
}

```

Insert Last

```

void insertlast (int val)
{
    struct node *newnode;
    newnode = (struct node*) malloc (sizeof
        (struct node));

```

```

    newnode->data = val;
    newnode->next = NULL;

```

```

    if (head == NULL)

```

```

    {
        head = newnode;
        head->next = head;
    }

```

```

    else

```

```

    {
        struct node *t = head;
        while (t->next != head)
            t = t->next;
        t->next = newnode;
        newnode->next = head;
    }
}

```

* C.LL insert middle & S.LL insert middle functions are same.

Delete First

```
void deleteFirst( )
```

```
{ if (head == NULL)
```

```
    printf("CLL is empty");
```

```
else if (head->next == head)
```

```
{
```

```
    struct node *t = head;
```

```
    head = NULL;
```

```
    free(t);
```

```
else
```

```
{
```

```
    struct node *p = head; *t = head;
```

```
    while (t->next != head)
```

```
        t = t->next;
```

```
    head = head->next;
```

```
    t->next = head;
```

```
    free(p);
```

```
}
```

```
}
```

Delete Last (a b c) (b c) (c)

void deleteLast()

{ if (head == NULL)

printf ("LL is empty");

else if (head → next == head)

{ struct node *t = head;

head = NULL;

free(t);

}

else

{ struct node *p, *t = head;

while (t → next != head)

{ p = t;

t = t → next;

}

p → next = head;

free(t);

}

Display

void display()

{ if (head == NULL)

printf ("LL is empty");

else {

struct node *t = head;

while (t → next != head)

{ printf ("%d", t → data);

t = t → next;

```
printf("%d", t->data);  
}  
}
```

03/02/2

Stack Implementation using Linked List

- By using stack we can insert & delete an element by $O(1)$ time complexity.
- Stack = First-In & Last-Out (FILO) (or) (LIFO)

Node structure

```
struct node
```

```
{  
    int data;
```

```
    struct node *next;
```

```
    *top = NULL;
```

Push

```
void push(int val)
```

```
{  
    struct node *newnode;
```

```
    newnode = (struct node*) malloc(sizeof  
        (struct node));
```

```
    newnode->data = val;
```

```
    newnode->next = top;
```

```
    top = newnode;
```

```
}
```

Pop

void Pop()

```
{
    if (top == NULL)
        printf("stack is empty");
    else
    {
        printf("Pop val: %d", top->data);
        struct node *t = top;
        top = t->next;
        free(t);
    }
}
```

Peek

void Peek()

```
{
    if (top == NULL)
        printf("stack is empty");
    else
        printf("Peek val: %d", top->data);
}
```

Display

void display()

```
{
    if (top == NULL)
        printf("stack is empty\n");
}
```

else

{

struct node *t = top;

while (t != NULL)

{
printf("v.d", t->data);

t = t->next;

}

printf("NULL");

}

}

Stack Applications

① Infix → Postfix conversion

② Infix → Prefix conversion

③ Post Fix evaluation

④ Balancing Parenthesis

⑤ String Reverse

⑥ Recursion

⑦ Infix → Postfix

Step 1:- Read infix expression

Step 2:- If character is 'C' then push into stack.

Step 3:- If the given char is operand add into postfix.

step 4:- If char is '(' then pop() upto 'C' open parenthesis. Add into post fix.

and discard '(' & ')'

step 5:- If char is operator

5.1:- If stack is empty or peek value is '(' then push operator into stack (or)

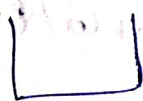
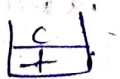
5.2:- Repeatedly check priority of char. Priority(char) < Priority(peek()) then pop (Add into postfix) then push the operator into stack.

step 6:- Repeat step no 2 to 5.

step 7:- End of the string pop all the character from stack and add into post fix.

Example

① "a + (b * c) * d - e / f"

①  top = -1 ② 'a' postfix = "a" ③ '+'  top = 0 ④ 'c'  top = 1	⑤ 'b' postfix = "ab" ⑥ '*'  top = 2 ⑦ 'c' postfix = "abc" ⑧ ')'  postfix = "abc*"
--	--

(x_i) —
 top = 0
 Pf $x = "abc*d?," +$

xiv. 'f'

Pfa = "abc * d y. + e f"

Pfa = "abc * d y. + e f /"

⑤

$\rightarrow \leq +$

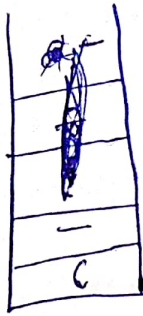
$\frac{\quad}{(}$

Pfa $> "ab +"$

$PFA = "ab+cd/"$
 ② 'e'
 $PFA = "ab+cd/e"$

①6 ')'
Pf x2 " a b t d l e f y . g *

17 -



Pfx = "ab+cd/ef%.g*--"

18

Pfx = "ab+cd/ef%.g*--h"

19

Pfx = "ab+cd/ef%.g*--h-"

Code for Infix to Postfix conversion

04/02/25

```
#include <stdio.h>
```

```
#define SIZE 100
```

```
int s[SIZE], top = -1;
```

```
void push(char ch)
```

```
{ s[++top] = ch;
```

```
}
```

```
char pop()
```

```
{ return s[top--];
```

```
}
```

```
char peek()
```

```
{ return s[top];
```

```
}
```

```
int priority(char ch)
```

```
{ if (ch == '(') return 1;
```

```
if (ch == '+' || ch == '-') return 2;
```

```
if (ch == '%' || ch == '*' || ch == '/') return 3;
```

```
if (ch == '\n') return 4;  
}
```

```
int main () {
```

```
char inf[100], pfx[100], tc;
```

```
int i, j = 0;
```

```
printf("enter infix: ");
```

```
scanf("%s", inf);
```

```
for(i=0; inf[i] != '\0'; i++) {
```

```
if (inf[i] == '(')
```

```
push(inf[i]);
```

```
else if (inf[i] >= 'a' & inf[i] <= 'z')
```

```
pfx[j++] = inf[i];
```

```
else if (inf[i] == ')') {
```

```
while ((tc = pop()) != '(')
```

```
pfx[j++] = tc;
```

```
}
```

```
else {
```

```
while (top != -1 & priority(inf[i]) <
```

```
priority(peek()))
```

```
pfx[j++] = pop();
```

```
push(inf[i]);
```

```
}
```

```
}
```

```

while (top != -1)
    pfx[i++] = pop();
    pfx[i] = '\0';
    printf("Postfix: %s", pfx);
    return 0;
}

```

② Postfix Evaluation

Step 1:- Read postfix expression.

Step 2:- If character is operand push into stack.

Step 3:- If character is operator then push (second - pop()) operator first - pop()

Step 4:- Repeat step 2 & 3 upto end of the string.

Step 5:- pop() → Final answer.

Ex:- $a + (b * c \cdot d) - e / f$

$1 + (2 * 3 \cdot 5) - 6 / 2$

$= 1 + 1 - 3$

$= 2 - 3$

$= -1 //$

1) $\boxed{} \text{ top} = -1$

2) $\boxed{a} \text{ top} = 0$

3) $\boxed{a, b} \text{ top} = 1$

4) $\boxed{a, b, c} \text{ top} = 2$

5) $*$
 $\text{Push}(b * c)$
 $\text{Push}(2 * 3)$
 $\boxed{a, b} \text{ top} = 1$

6) $\boxed{a, 6, d}$

7) $\%$
 $\text{Push}(6 * d)$
 $\text{Push}(6 * 5)$

$\boxed{a, 1} \text{ top} = 1$

8) $+$
 $\text{Push}(a + 1)$
 $\text{Push}(1 + 1)$

$\boxed{2} \text{ top} = 0$

9) $/$
 $\boxed{2, 6} \text{ top} = 1$

$\boxed{2, 6, e, f}$

11) $/$
 $\text{Push}(e / f)$
 $\text{Push}(6 / 2)$

$\boxed{3} \text{ top} = 1$

12) $-$
 $\text{Push}(2 - 3)$

$\boxed{-1} \text{ top} = 0$

$= -1 //$

Ex- $ab + cd / ef \% g * - - h -$

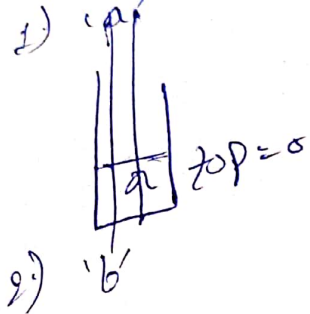
$a=2, b=3, c=6, d=4, e=2, f=7, g=4, h=1$

~~$= 2 \times 3 + 6 \times 4 / 2 \times 7 \% 4 * - - 1 -$~~

~~$= 6 + 24 / 14 \% 4 * - - 1 -$~~

~~$= 6 + 1 \% 4 * - - 1 -$~~

~~$= 6 + 1 * - - 1 -$~~



$$(a + b - (c / d - e \% f * g) - h)$$

$$(2 + 3 - (6 / 4 - 2 \% 7 * 4) - 1)$$

$$(2 + 3 - (1 - 2 * 4) - 1)$$

$$(2 + 3 - (-7) - 1)$$

$$(5 + 7 - 1)$$

$$(12 - 1)$$

$$= 11$$

05/02/25

Code for postfix evaluation

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 100
int s[SIZE], top = -1;
void push (int x) {
    s[++top] = x;
}
int pop () {
    return s[top--];
}
int main () {
    char pfx[100];
    int i, first, second;
    printf("Enter postfix: ");
    scanf("%s", pfx);
    for (i = 0; pfx[i] != '\0'; i++) {
        if (pfx[i] >= 'a' & pfx[i] <= 'z') {
            printf("Enter %c value: ", pfx[i]);
            scanf("%d", &val);
            push(val);
        }
    }
}
```

```

    }
    else
    {
        first = pop();
        second = pop();
        switch (pfal[i]) {
            case '+':
                push(second + first);
                break;
            case '-':
                push(second - first);
                break;
            case '*':
                push(second * first);
                break;
            case '/':
                push(second / first);
                break;
            case '%':
                push(second % first);
                break;
            default:
                exit(0);
        }
    }
}

```

```

}
printf("PostFix Evaluation: %d", pop());
return 0;
}

```

Ex:- $a b c d * e - + g h * + - * + 1 -$

$a = 2$

$b = 3$

$c = 4$

$d = 5$

$e = 1$

$f = 6$

$g = 2$

$h = 3$

$i = 4$

1.) $\boxed{} \text{ top} = -1$

2.) $\boxed{a} \text{ top} = 0$

3.) $\boxed{b \over a} \text{ top} = 1$

4.) $\boxed{c \over b \over a} \text{ top} = 2$

5.) $\boxed{d \over c \over b \over a} \text{ top} = 3$

6.) $*$
 $\text{push}(c * d)$
 $\text{push}(4 * 5)$
 $\text{push}(20)$

7.) e
 $\boxed{e \over 20 \over b \over a}$

8.) $-$
 $\text{push}(20 - e)$
 $\text{push}(19)$

$\boxed{19 \over b \over a}$

9.) f
 $\boxed{19 \over b \over a} \text{ top} = 2$

10.) g
 $\boxed{19 \over b \over a} \text{ top} = 3$

11.) h
 $\boxed{19 \over b \over a} \text{ top} = 4$

12.) $*$
 $\text{push}(g * h)$
 $\text{push}(2 * 3)$
 $\text{push}(6)$

$\boxed{6 \over 19 \over b \over a}$

$\boxed{-44}$

16.) $+$

13.) $+$
 $\text{push}(19 + 6)$
 $\text{push}(25)$

$\boxed{25 \over b \over a} \text{ top} = 2$

14.) $-$
 $\text{push}(b - 25)$
 $\text{push}(3 - 25)$
 $\text{push}(-22)$

$\boxed{-22 \over a}$

Balancing Parentheses

Algorithm:- ① Read parentheses symbol.

② If the character is ~~open~~ '(', '{' or '[' push into stack.

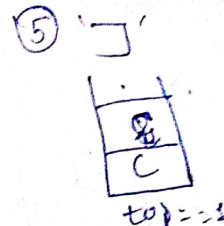
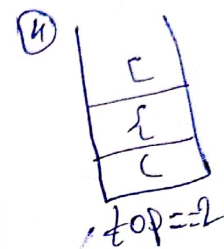
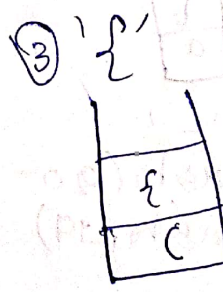
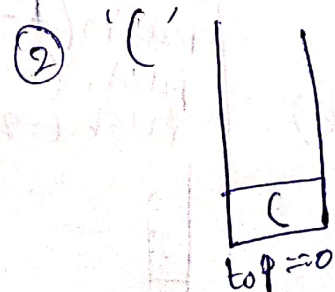
③ If the character is ')', '}' or ']'

3.1 To check stack is empty or not
3.1.1 If stack is empty print unbalanced parentheses.

3.1.2 To check the current character is ')' and '}' and ']' and Peek() == '(' (or) '[' and Peek() == '[' then pop.

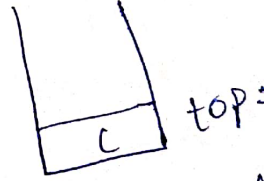
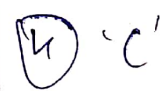
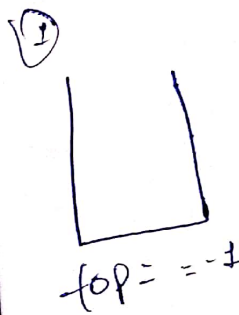
④ At the end of the string to check stack is empty then print balanced parentheses.

Ex:- ({ [] })



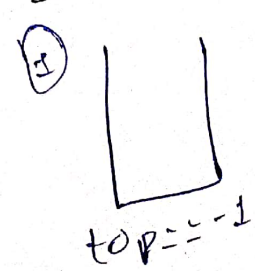
∴ Balanced Parentheses

Ex:- () { }

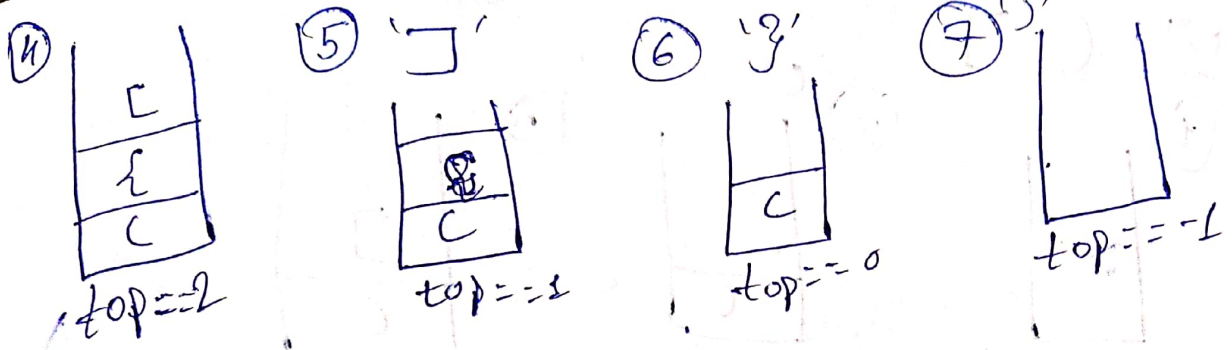


∴ Unbalanced

Ex:- () }

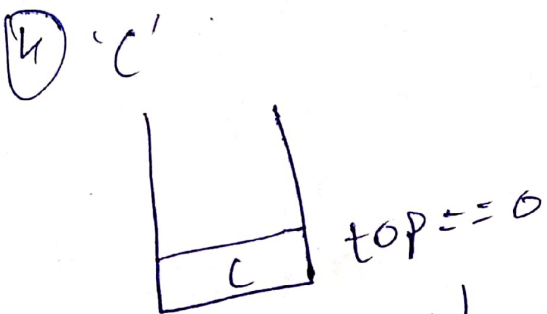
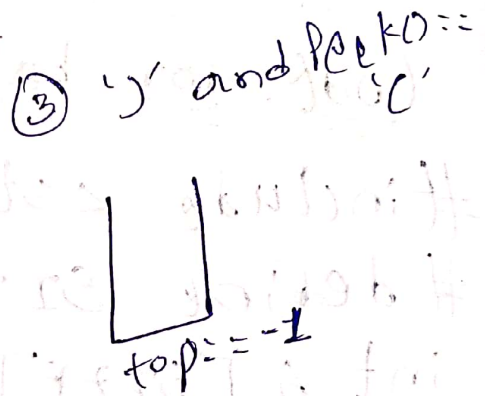
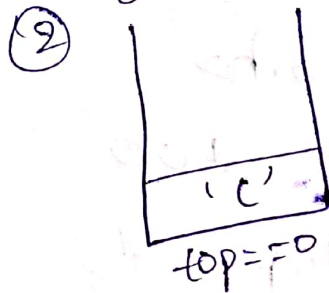
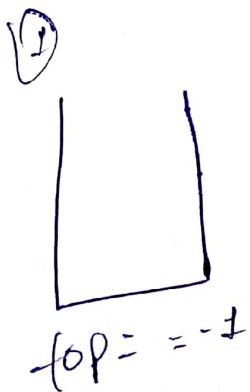


∴ Unbalanced



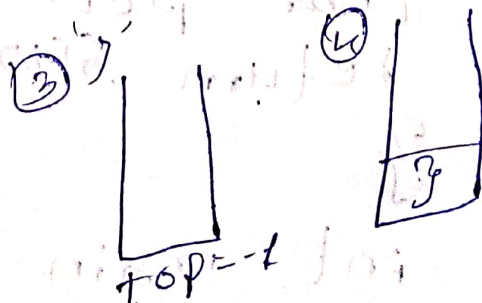
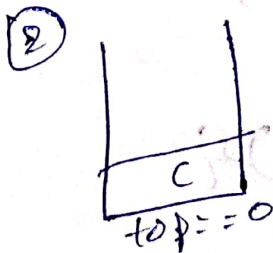
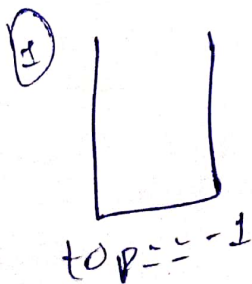
∴ Balanced Parentheses.

Ex:- () (



∴ Unbalanced parentheses

Ex:- () }



∴ Unbalanced parentheses.

EX - []

①

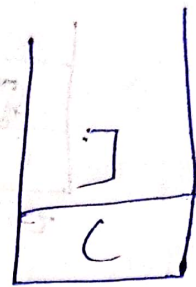


top = -1

②



③



∴ Unbalanced

Code for balancing parenthesis:

```
#include <stdio.h>
#define SIZE 100
int s [SIZE], top = -1;
void push (char ch) {
    s[++top] = ch;
```

```
}
```

```
void pop () {
    s[top--];
```

```
}
```

```
char peek () {
    return s[top];
```

```
}
```

```
int main () {
```

```
    char p[100];
```

```
    int i;
```

```
    scanf ("%s", p);
```

```
for
```

```

for (i=0; p[i] != '\0'; i++) {
    if (p[i] == '(' || p[i] == '[' || p[i] == '{')
        push(p[i]);
    else if (top != -1 && ((p[i] == ')' &&
        peek() == '(') || (p[i] == ']' &&
        peek() == '[') || (p[i] == '}' &&
        peek() == '{')))
        pop();
    else
        break;
}
if (p[i] == '\0' && top == -1)
    printf("Balanced Parenthesis");
else
    printf("unbalanced Parenthesis");
return 0;
}

```

Queue using linked list

→ It follows FIFO property.

Queue node structure

```
struct node
```

```
{ int data;
```

```
  struct node *next;
```

```
} *front = NULL, *rear = NULL;
```

```
void enqueue(int val)
```

```
{ struct node *newnode;
```

```
  newnode = (struct node*) malloc(sizeof  
    (struct node));
```

```
  newnode → data = val;
```

```
  newnode → next = NULL;
```

```
  if (front == NULL)
```

```
    front = rear = newnode;
```

```
  else
```

```
  {
```

```
    rear → next = newnode;
```

```
    rear = newnode;
```

```
  }
```

```
void dequeue()
```

```
{ if (front == NULL)  
  printf ("Queue is empty");
```

```
else
```

```
{ struct node *t = front;  
  front = front->next;  
  if (front == NULL)  
    rear = NULL;  
  free(t);
```

```
}
```

```
}
```

```
void display()
```

```
{ if (front == NULL)  
  printf ("Queue is empty");
```

```
else
```

```
{ struct node *t = front;  
  while (t != NULL)
```

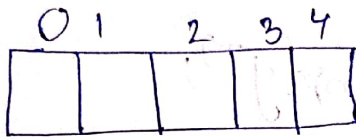
```
{ printf ("%d", t->data);  
  t = t->next;
```

```
}
```

```
}
```

```
}
```

Circular Queue



f = r = -1

1) enq(10)

2) enq(20)

3) enq(30)

4) deq()

5) enq(40)

6) enq(50)

7) deq()

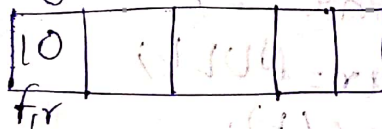
8) enq(60)

1) if (f == -1)

f++

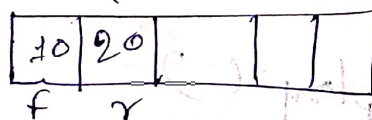
$r = (r + 1) \% \text{size}$

$q[r] = 10$



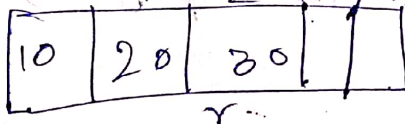
2) $r = (0 + 1) \% 5 = 1$

$q[1] = 20$



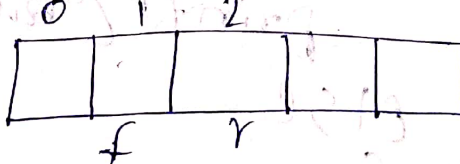
3) $r = (1 + 1) \% 5 = 2$

$q[2] = 30$



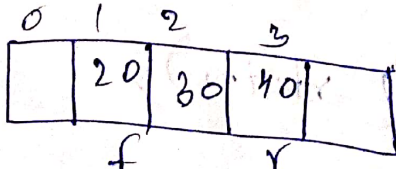
4) $f = (f + 1) \% \text{size}$

$f = (0 + 1) \% 5 = 1$



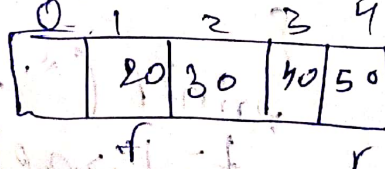
5) $r = (2 + 1) \% 5 = 3$

$q[3] = 40$

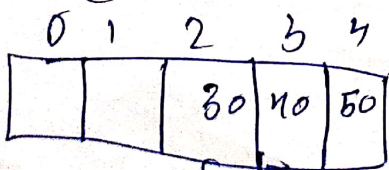


6) $r = (3 + 1) \% 5 = 4$

$q[4] = 50$

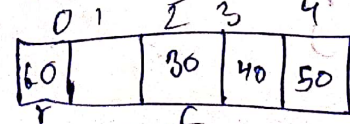


7) $f = (1 + 1) \% 5 = 2$



8) enq(60)

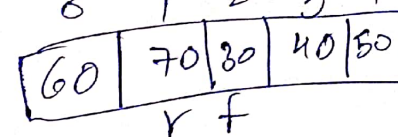
$r = (4 + 1) \% 5 = 0$



9) enq(70)

$r = (0 + 1) \% 5 = 1$

$q[1] = 70$



Code for circular

```
#include <stdio.h>
```

```
#define SIZE 5
```

```
int q[SIZE]
```

```
void enqueue
```

```
{ if (rear +
```

```
printf(
```

```
else {
```

```
if (front
```

```
front
```

```
rear
```

```
q[rear]
```

```
}
```

```
}
```

```
void dequeue
```

```
{ if (front
```

```
printf(
```

```
else {
```

```
printf(
```

```
if (front
```

```
front
```

```
}
```

⑨ enqueue(40)

$r = (0 + 1) \% 5 = 1$ (30) enqueue(80)

$q[1] = 40$

Q is overflow

0	1	2	3	4
60	70	30	40	50
	r	f		

Code for circular queue

```
#include <stdio.h>
```

```
#define SIZE 5
```

```
int q[SIZE], front = -1, rear = -1;
```

```
void enqueue (int val) {
```

```
    if ((rear + 1) % SIZE == front)
```

```
        printf("Queue is overflow\n");
```

```
    else {
```

```
        if (front == -1)
```

```
            front++;
```

```
        rear = (rear + 1) % SIZE;
```

```
        q[rear] = val;
```

```
    }
```

```
}
```

```
void dequeue ( ) {
```

```
    if (front == -1)
```

```
        printf("Queue is underflow\n");
```

```
    else {
```

```
        printf("dequeue val : %d\n", q[front]);
```

```
        if (front == rear)
```

```
            front = rear = -1;
```

```
    }
```

```

else {
    front = (front + 1) % SIZE;
}
}

```

```

void display () {
    if (front == -1)
        printf("Queue is empty\n");
    else {
        int i = front;
        while (i != rear) {
            printf("%d ", a[i]);
            i = (i + 1) % SIZE;
        }
        printf("%d\n", a[rear]);
    }
}

```

```

}

int main () {
    int option, val;
    while (1) {
        scanf("%d", &option);
        printf("1. enq 2. deq 3. display 4. exit\n");
        switch (option) {
            case 1: scanf("%d", &val);
                    enqueue(val);

```

case 2: dequeue();

case 3: scanf("%d", &val);
display;

case 4: exit(0)

default: printf("Enter a valid number")

}

}

Q: Consider queue size is 4 & perform following operations.

deq();

enq(10);

enq(20);

enq(30);

enq(40);

enq(50);

deq();

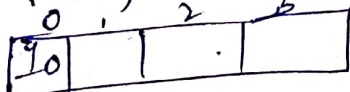
enq(60);

① deq();

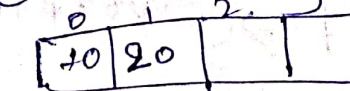
queue is underflow

② enq(10);

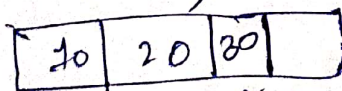
f++;



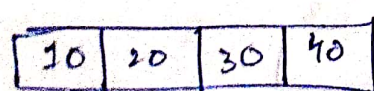
③ enq(20)



④ enq(30);



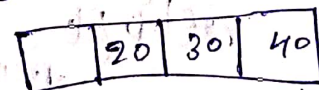
⑤ enq(40);



⑥ enq(50);

queue is overflow

⑦ deq();



⑧ ~~enq(10);~~
deq;



⑨



s_1 s_2 s_3
8 9 7

5 5 7

5 5 6

5 5 5

Double Ended Queue

- In DE queue's we can insert and delete from front and rear index.
- (i) DE queue implementation using circular queues.
 - (ii) DE queue implementation using DLL



1) enq (60)

0	1	2	3
10	20	30	40

$$(r+1) \% \text{SIZE} = f \Rightarrow (\text{Overflow})$$

2) enq (10)

$f = -1$

$r = -1$

if ($f == -1$)
 $r = -1$

0	1	2	3

if ($f == -1$)

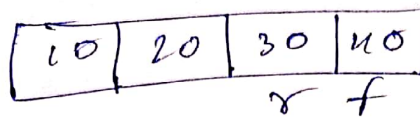
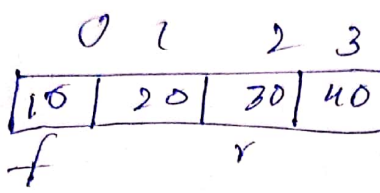
$f = r = 0$

0	1	2	3
10			

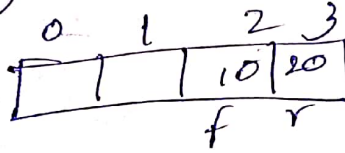
③ $enq(30)$

if ($f == 0$)

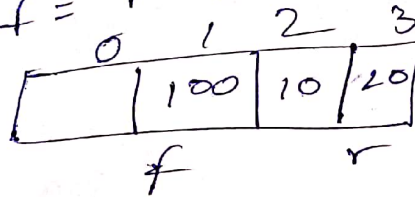
$f = SIZE - 1$



④ $enq(100)$



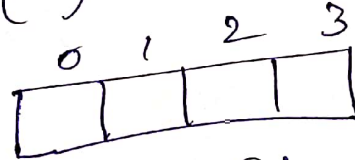
$f = f - 1$



and
ex.

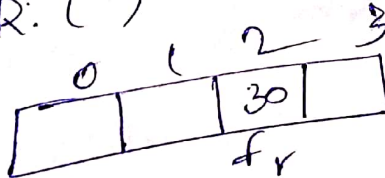
1) $deq R()$

$f = -1$
 $r = -1$



"underflow"

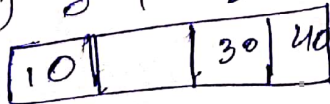
2) $deq R()$



if ($f == r$)

$f = r = -1$

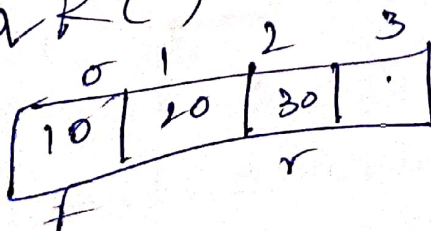
3) $deq R()$



if ($r == 0$)

$r = SIZE - 1$

4) $deq R()$



$r = r - 1$

